USART Driver Analysis

The USART (Universal Synchronous Asynchronous Receiver Transmitter) driver is used for serial communication between the STM32F446xx microcontroller and external devices such as a PC, sensors, or other microcontrollers. It provides an easy interface for configuring the USART peripheral and sending data without directly accessing the hardware registers.

The driver is implemented using two files:

- **stm32f446xx_usart_driver.h** – Contains structures, macros, and function declarations.
- **stm32f446xx_usart_driver.c** – Contains the implementation of USART functions.

This driver supports baud rate configuration, transmitter and receiver selection, parity control, stop bits, hardware flow control, and data transmission.

How the USART Driver Works

The USART driver first enables the clock for the selected USART peripheral. After enabling the clock, it configures the communication parameters such as baud rate, word length, stop bits, parity, and hardware flow control.

The baud rate is calculated using the peripheral clock value obtained from the RCC driver. The calculated value is then written into the BRR (Baud Rate Register).

When the application wants to send data, it simply calls the transmit function. The driver waits until the transmit buffer becomes empty, writes the data into the Data Register (DR), and continues until all bytes are transmitted.

How the Driver Hides Hardware Complexity

Without a driver, the programmer has to manually configure several USART registers such as CR1, CR2, CR3, BRR, SR, and DR.

The USART driver hides these low-level register operations by providing simple APIs.

Instead of writing multiple register values, the application only needs to create a configuration structure and call:

USART_Init(&USARTHandle);

USART_SendData(&USARTHandle, message, length);

The driver automatically performs all register configurations internally, making the code easier to understand and maintain.

USART Configuration Structures

USART_Config_t

This structure stores all USART configuration parameters.

It contains:

- USART mode (TX/RX/TXRX)
- Baud rate
- Number of stop bits
- Word length
- Parity control
- Hardware flow control

USART_Handle_t

This structure combines the USART peripheral address and configuration settings.

It contains:

- Pointer to USART peripheral
- USART configuration structure

The handle is passed to all driver functions.

APIs Provided by the Driver

1. USART_PeriClockControl()

This function enables the clock for the selected USART peripheral.

Without enabling the clock, the USART registers cannot be accessed.

2. USART_SetBaudRate()

This function calculates the baud rate register value using the APB clock frequency obtained from the RCC driver.

It supports both OVER8 and OVER16 oversampling modes.

The calculated value is stored in the BRR register.

3. USART_Init()

This function initializes the USART peripheral.

It performs the following configurations:

- Enables peripheral clock
- Configures transmitter or receiver mode
- Sets word length
- Configures parity
- Sets stop bits
- Configures hardware flow control
- Calculates and sets baud rate

This API prepares the USART peripheral for communication.

4. USART_PeripheralControl()

This function enables or disables the USART peripheral by setting or clearing the UE (USART Enable) bit in the CR1 register.

When enabled, the USART becomes ready for transmission and reception.

5. USART_GetFlagStatus()

This function checks the status flags available in the Status Register (SR).

It is mainly used to monitor:

- TXE (Transmit Data Register Empty)
- RXNE (Receive Data Register Not Empty)
- TC (Transmission Complete)

The function returns SET or RESET depending on the flag status.

6. USART_SendData()

This function transmits data from the microcontroller.

Working steps:

1. Wait until the TXE flag becomes SET.
2. Write one byte into the Data Register (DR).
3. Repeat until all bytes are transmitted.
4. Wait until the TC (Transmission Complete) flag becomes SET.

This ensures that the complete message is successfully transmitted.

Driver Features

The USART driver supports:

- Configurable baud rate
- Transmit mode
- Receive mode
- Full duplex communication
- Even and odd parity
- One stop bit configuration
- Hardware flow control (CTS/RTS)
- Status flag monitoring
- Blocking data transmission

Advantages of the USART Driver

- Simple API-based programming
- No direct register manipulation in application code
- Easy baud rate configuration
- Modular and reusable design
- Improves readability and maintainability
- Easy integration with other drivers such as RCC and GPIO

The USART driver provides a simple and organized interface for serial communication in the STM32F446xx microcontroller. It hides the complexity of register-level programming and allows the application to configure and use the USART peripheral through easy-to-use APIs. The driver supports essential communication features such as baud rate calculation, transmission, parity control, and hardware flow control, making it suitable for embedded system applications.